

Z5008 Big Data Lab

Real-Time Retail Analytics & Demand Prediction Platform

Comprehensive Project Documentation

Author: Vineet Joshi

Roll Number: ZDA25M007

Institution: IIT Madras Zanzibar

Course: Z5008 Big Data Lab

Supervisor: Dr. Innocent Nyalala

Even Semester 2026

Project Overview

Objective

This project builds a production-grade, end-to-end Big Data system that ingests real-time retail transaction data, stores it in a Lakehouse architecture, processes it with Apache Spark, trains machine learning models, deploys them as REST APIs, and monitors everything with live dashboards.

Domain

Retail Analytics — using the Online Retail II dataset from UCI Machine Learning Repository containing UK-based online retail transactions from 2009 to 2011.

Dataset

- Source: Online Retail II (UCI ML Repository)
- Original records: 500,000+ transactions
- Scaled dataset: 15.3 million records (retail_20m.parquet)
- Revenue: GBP 10.88 million total
- Countries: 40 markets served
- Customers: 4,649 unique customers
- Products: 4,045 unique SKUs

8-Layer Architecture

Layer	Component	Tool Used	Status
1	Data Ingestion	Apache Kafka 3.7 (KRaft)	Done - NB3
2	Object Storage	MinIO + Delta Lake	Done - NB4
3	Batch Processing	Apache Spark 3.5.3	Done - NB5
4	Orchestration	Apache Airflow 2.9.1	Done - NB12
5	ML Training	Scikit-learn + MLflow	Done - NB6
6	Experiment Tracking	MLflow v2.11.0	Done - NB6
7	Model Serving	Flask REST API	Done - NB10
8	Monitoring	Grafana + Prometheus	Done - NB11
BONUS	NL Query Interface	Groq Llama 3 + DuckDB	Done - NB13

Notebook 1: Setup and Exploratory Data Analysis

Objective

Set up the project environment, load the Online Retail II dataset, perform comprehensive exploratory data analysis (EDA), and prepare clean data for downstream processing.

What This Notebook Does

- Connects to MinIO object storage and creates the retail-v2 bucket
- Loads the Online Retail II CSV file (online_retail_II.csv) from the Jupyter filesystem
- Performs data cleaning: removes nulls, negative quantities, zero prices, and invalid records
- Creates derived columns: Revenue, Hour, DayOfWeek, DayName, Month, Year, WeekOfYear
- Generates EDA visualizations: revenue distribution, top countries, product popularity, time patterns
- Saves cleaned data as retail_clean.parquet for use in subsequent notebooks

Key Results

- Raw records loaded: 1,067,371 rows
- Clean records after filtering: 500,000+ valid transactions
- Total revenue: GBP 10.88 million
- Date range: December 2009 to December 2011
- Top market: United Kingdom (83% of revenue)

Technologies Used

- Python: pandas, numpy, matplotlib, seaborn
- Storage: MinIO (S3-compatible) via boto3
- Output: retail_clean.parquet

Notebook 2: Data Scaling to 15 Million Records

Objective

Scale the cleaned dataset from 500K records to 15+ million records to demonstrate Big Data 5V characteristics: Volume, Velocity, Variety, Veracity, and Value.

What This Notebook Does

- Reads retail_clean.parquet (500K records) and applies chunk-based data augmentation
- Adds realistic noise and variation to each chunk: price fluctuations, quantity variations, temporal shifts
- Writes the scaled dataset to retail_20m.parquet using fastparquet for efficient storage
- Validates all 5 Big Data V's with measurements and evidence
- Reports file size and record count for verification

Key Results

- Output records: 15.3 million rows (target was 10-20M)
- File size: 577 MB (retail_20m.parquet)
- Volume: 15.3M records verified
- Velocity: 2,000 records per second streaming rate
- Variety: 40 countries, 4,045 products, multiple data types
- Veracity: Data quality checks passed
- Value: GBP 10.88M revenue insights extracted

Technologies Used

- Python: pandas, numpy, fastparquet
- Storage: Local filesystem

Notebook 3: Kafka Real-Time Streaming

Objective

Set up Apache Kafka to stream retail transaction data in real-time, simulating a live retail system where transactions are produced and consumed continuously.

What This Notebook Does

- Connects to Kafka broker at kafka:9092 running in KRaft mode (no Zookeeper)
- Creates topic retail-txn-v2 with 3 partitions for parallel processing
- Implements a Kafka producer that streams 500,000 records at 2,000 records/second
- Uses kafka-python library for reliable message production
- Verifies message delivery with consumer confirmation
- Reports throughput, latency, and partition distribution metrics

Key Results

- Topic: retail-txn-v2 (3 partitions, KRaft mode)
- Records streamed: 500,000
- Throughput: 2,000 records/second
- Broker: kafka:9092 (Kafka 3.7)
- Mode: KRaft (no Zookeeper dependency)

Technologies Used

- Apache Kafka 3.7 (KRaft mode)
- kafka-python library
- JSON serialization for messages

Notebook 4: Delta Lake Storage

Objective

Consume the Kafka stream and persist retail transactions to Delta Lake tables on MinIO, providing ACID-compliant, versioned storage with schema enforcement.

What This Notebook Does

- Consumes messages from Kafka topic retail-txn-v2 using kafka-python consumer
- Converts JSON messages to pandas DataFrames with proper type casting
- Writes data to Delta Lake tables on MinIO bucket retail-v2 using the deltalake library
- Creates multiple Delta tables: transactions, features, predictions, aggregations
- Validates data integrity with record counts and schema checks
- Reports storage metrics including file sizes and partition counts

Key Results

- Records written: 500,000 to s3://retail-v2/delta/transactions
- Features computed: 272,000 feature rows
- Countries covered: 40
- Revenue stored: GBP 10.8 million
- Storage format: Delta Lake (Apache Parquet + transaction log)
- MinIO bucket: retail-v2

Technologies Used

- Delta Lake (deltalake Python library)
- MinIO (S3-compatible object storage)
- kafka-python (consumer)
- PyArrow for columnar data operations

Notebook 5: Apache Spark Batch Processing

Objective

Use Apache Spark to perform distributed batch processing on the retail data, computing complex aggregations, window functions, and writing results back to Delta Lake.

What This Notebook Does

- Initializes Spark in local[*] mode connecting to the Spark cluster
- Loads 805,000 records from Delta Lake into Spark DataFrames
- Applies window functions: RANK, LAG, rolling 7-day moving average
- Computes country-level revenue aggregations with ranking
- Computes daily revenue trends with moving averages
- Computes monthly growth rates using LAG function
- Writes 4 aggregated Delta tables back to MinIO retail-v2 bucket
- Executes 3 Spark SQL queries for distributed analytics

Key Results

- Records processed: 805,000
- Window functions used: RANK, LAG, ROLLING AVG
- Delta tables written: spark_features, spark_country_agg, spark_daily_ma, spark_monthly
- Spark SQL queries: 3 (feature stats, daily MA, monthly growth)
- Spark version: 3.5.3 (bitnami/spark)

Technologies Used

- Apache Spark 3.5.3
- PySpark (DataFrame API and SQL)
- Delta Lake integration

- MinIO S3 storage backend

Notebook 6: Machine Learning Model Training

Objective

Train multiple machine learning models to predict retail demand (TotalQuantity) from engineered features, track experiments with MLflow, and select the best performing model.

What This Notebook Does

- Loads feature data from Delta Lake (retail-v2/delta/features)
- Engineers 10 input features from transaction data
- Trains 3 ML models: Linear Regression, Random Forest, Gradient Boosting
- Tracks all experiments with MLflow (local file tracking)
- Evaluates models using R2 score and RMSE metrics
- Selects the best model based on R2 score
- Saves the best model as model_bundle.joblib with encoders and feature list

Features Used

Feature	Description	Type
Year	Year of transaction	Integer
Month	Month number 1-12	Integer
WeekOfYear	Week number 1-52	Integer
DayOfWeek	Day 0=Monday 6=Sunday	Integer
AvgUnitPrice	Average price per unit	Float
NumInvoices	Number of invoices	Integer
UniqueCustomers	Unique customer count	Integer
TotalRevenue	Total revenue in GBP	Float
StockCode_idx	Encoded product code	Integer
Country_idx	Encoded country	Integer

Model Results

Model	R2 Score	RMSE	Status
Linear Regression	0.9289	87.97	BEST MODEL
Random Forest	0.8912	102.45	Good
Gradient Boosting	0.8756	109.23	Good

Key Results

- Best model: Linear Regression (R2 = 0.9289)
- Training set: 218,000 rows
- Test set: 54,000 rows
- MLflow experiment: retail-demand-prediction-v2
- Model saved: /home/jovyan/work/model_bundle.joblib

Technologies Used

- Scikit-learn (LinearRegression, RandomForestRegressor, GradientBoostingRegressor)
- MLflow for experiment tracking
- joblib for model serialization

Notebook 7: Model Evaluation and Predictions

Objective

Comprehensively evaluate the trained ML model with residual analysis, visual comparisons of actual vs predicted values, and persist predictions to Delta Lake for downstream use.

What This Notebook Does

- Loads the best model (Linear Regression) from model_bundle.joblib
- Generates predictions on the test set (54,507 rows)
- Creates residual plot to check for systematic errors
- Creates actual vs predicted scatter plot to visualize model fit
- Creates weekly actual vs predicted time series comparison
- Writes all predictions to Delta Lake (s3://retail-v2/delta/predictions)
- Exports model_bundle.joblib with model, encoders, and features

Key Results

- Predictions generated: 54,507 rows
- R2 Score confirmed: 0.9289 (92.89% variance explained)
- RMSE confirmed: 87.97 units
- Predictions stored: s3://retail-v2/delta/predictions
- Residuals: normally distributed around zero (good sign)

Technologies Used

- Scikit-learn for predictions
- Matplotlib for visualization
- Delta Lake for prediction storage
- joblib for model loading

Notebook 8: SQL Analytics with DuckDB and Spark SQL

Objective

Perform comprehensive business analytics using both DuckDB (for fast in-memory SQL) and Spark SQL (for distributed queries), covering 13 different analytical queries across all major business dimensions.

What This Notebook Does

- Loads transaction, features, and predictions data from Delta Lake
- Executes 10 DuckDB business analytics queries
- Executes 3 Spark SQL distributed queries
- Demonstrates advanced SQL: GROUP BY, Window Functions, CTEs, CASE WHEN, LAG, RANK, Pareto

Queries Executed

#	Query Type	SQL Features Used
1	Country Revenue Analysis	GROUP BY, SUM, ORDER BY
2	Monthly Revenue Trend	GROUP BY, DATE functions
3	Best-Selling Products	GROUP BY, SUM, LIMIT
4	Hourly Sales Pattern	GROUP BY Hour, AVG
5	Customer Segmentation (RFM)	GROUP BY, COUNT DISTINCT

#	Query Type	SQL Features Used
6	Day-of-Week Analysis	GROUP BY DayName
7	Prediction Accuracy by Country	JOIN, ROUND
8	Pareto Analysis	CTE, cumulative SUM
9	Seasonal Demand Patterns	CASE WHEN, GROUP BY
10	Cross-Country Product Ranking	Window RANK() OVER
11	Spark SQL Feature Statistics	DESCRIBE, aggregations
12	Spark SQL Daily Moving Avg	Window AVG() OVER
13	Spark SQL Monthly Growth	Window LAG() OVER

Technologies Used

- DuckDB for fast in-memory SQL analytics
- Apache Spark SQL for distributed queries
- Delta Lake as the data source

Notebook 9: Real-Time Interactive Dashboard

Objective

Build a live, auto-refreshing analytics dashboard using Plotly Dash that visualizes all key retail metrics in real-time with a dark professional theme, including an interactive ML demand predictor.

What This Notebook Does

- Loads transaction data from Delta Lake into memory
- Pre-computes aggregations: daily revenue, country breakdown, top products, heatmap, monthly trends
- Builds a Plotly Dash application with dark theme
- Implements auto-refresh every 3 seconds via dcc.Interval
- Creates 5 interactive charts with real data
- Implements ML demand predictor widget using trained Linear Regression model
- Starts live Kafka stream simulator showing real-time message counts

- Launches dashboard on port 8050

Dashboard Components

Component	Type	Description
Revenue KPI	Stat Card	GBP 10.88M total revenue
Transactions KPI	Stat Card	22,333 unique invoices
Customers KPI	Stat Card	4,649 unique customers
AOV KPI	Stat Card	Average order value
Products KPI	Stat Card	4,045 unique SKUs
Countries KPI	Stat Card	40 markets served
Daily Revenue Trend	Line Chart	Revenue with 7-day MA
Revenue by Country	Bar Chart	Top 15 countries
Sales Heatmap	Heatmap	Day x Hour revenue matrix
Top Products	Bar Chart	Top 10 by revenue
Monthly Revenue	Dual-axis	Revenue + quantity
Demand Predictor	Widget	ML prediction form
Kafka Monitor	Stats Bar	Live stream metrics

Key Results

- Dashboard URL: <http://localhost:8050>
- Auto-refresh: every 3 seconds
- Live blinking LIVE indicator
- Demand predictor: 125 units predicted for November Peak Season

Technologies Used

- Plotly Dash 2.17.0 with Bootstrap dark theme
- Plotly 5.20.0 for charts
- Delta Lake for data source
- Threading for background Kafka simulator

Notebook 10: REST API for Model Serving

Objective

Deploy the trained ML model as a production-ready REST API with an interactive web UI, accepting POST requests for demand predictions, with full load testing to demonstrate system stability.

What This Notebook Does

- Loads the trained Linear Regression model from `model_bundle.joblib`
- Creates a Flask REST API with 4 endpoints
- Adds an interactive web UI at the root endpoint
- Starts the API server on port 3001 in a background thread
- Tests all endpoints with real data
- Runs 10x load test with latency measurements
- Runs 100x extended load test with P50/P95/P99 latency
- Creates Dockerfile and requirements.txt for containerization

API Endpoints

Method	Endpoint	Description	Input
GET	/	Interactive web UI with prediction form	Browser
POST	/predict	Single demand prediction	JSON object
POST	/batch_predict	Multiple predictions	JSON array
GET	/health	Health check	None
GET	/model_info	Model metadata	None

Load Test Results

- 10x load test: 100% success rate
- 100x load test: 100% success rate
- Average latency: under 50ms per request
- Throughput: 20+ requests per second
- System stability: STABLE under load

Technologies Used

- Flask (REST API framework)
- joblib (model loading)
- Docker (containerization)
- Threading (background server)

Notebook 11: Grafana Monitoring Dashboard

Objective

Set up comprehensive system monitoring using Prometheus metrics and Grafana dashboards, tracking all pipeline components in real-time including Kafka throughput, Delta Lake records, API performance, and ML model metrics.

What This Notebook Does

- Creates a Prometheus metrics endpoint on port 8099
- Defines 16 custom metrics covering all pipeline layers
- Updates metrics every 5 seconds in a background thread
- Creates prometheus.yml configuration for scraping
- Generates Grafana dashboard JSON with 11 panels
- Imports dashboard to Grafana via REST API
- Configures Prometheus as Grafana data source

Metrics Tracked

Metric	Type	Description
kafka_messages_total	Counter	Total messages streamed
kafka_throughput_per_sec	Gauge	Messages per second
delta_lake_records_total	Gauge	Total Delta Lake records
api_requests_total	Counter	Total API predictions
api_latency_seconds	Histogram	API response latency
ml_model_r2_score	Gauge	R2 score (0.9289)
ml_model_rmse	Gauge	RMSE (87.97)
pipeline_revenue_total	Gauge	Total revenue GBP
pipeline_customers_total	Gauge	Unique customers
system_cpu_percent	Gauge	CPU usage
system_memory_percent	Gauge	Memory usage

Grafana Dashboard

- URL: http://localhost:3002 (admin/admin)
- Dashboard: Retail Analytics Pipeline - Z5008
- Panels: 11 (stat cards, gauges, timeseries)
- Auto-refresh: every 5 seconds
- Live metrics: Kafka 288K messages, 368 msg/sec, GBP 10.9M revenue, R2 = 0.9289

Technologies Used

- Prometheus (metrics collection)
- Grafana (visualization)
- prometheus_client Python library
- Flask (metrics endpoint)

Notebook 12: Airflow Pipeline Orchestration

Objective

Create and deploy an Apache Airflow DAG that orchestrates the entire retail analytics pipeline, scheduling all 8 tasks to run automatically every 6 hours with proper dependency management.

What This Notebook Does

- Creates retail_analytics_dag.py with 8 pipeline tasks
- Copies the DAG file to the Airflow container
- Verifies DAG appears in Airflow UI
- Triggers a manual DAG run via Airflow REST API
- Confirms all 8 tasks complete successfully

DAG Pipeline Flow

Task ID	Description	Checks
check_data_ingestion	Verify Delta Lake records and Kafka topic	Record count > 100K
run_data_quality_check	Validate data quality with 5 checks	Quality score > 80%
run_spark_processing	Execute Spark batch aggregations	6 processing steps
run_feature_engineering	Generate 10 ML features	Feature count = 10
run_ml_training	Load and validate trained model	Model R2 >= threshold

Task ID	Description	Checks
run_model_validation	Check R2 >= 0.80 and RMSE <= 200	Pass/Fail gate
check_api_health	Verify REST API is running	HTTP 200 response
generate_pipeline_report	Create summary report with all metrics	Report generated

Key Results

- DAG ID: retail_analytics_pipeline
- Schedule: Every 6 hours (timedelta(hours=6))
- Tags: retail, z5008, bigdata, mlops
- Total run duration: 13 seconds
- Status: Total success (all 8 tasks green)
- Airflow UI: http://localhost:8090 (admin/admin)

Technologies Used

- Apache Airflow 2.9.1
- PythonOperator for all tasks
- XCom for inter-task data passing
- Airflow REST API for programmatic control

Notebook 13: Natural Language Query Interface (Bonus +5%)

Objective

Build a natural language interface that allows users to query retail data by typing plain English questions. Groq Llama 3 AI translates the questions to SQL, which runs on Delta Lake data via DuckDB, with SQL injection prevention and read-only access enforcement.

What This Notebook Does

- Loads transaction data from Delta Lake into DuckDB (in-memory)
- Implements SQL safety layer blocking all write operations
- Uses Groq Llama 3 (llama-3.3-70b-versatile) to translate English to SQL
- Tests 5 natural language queries end-to-end
- Tests SQL injection prevention with malicious inputs
- Builds interactive web UI at port 8051

- Stores query history for review

Security Features

Security Layer	Implementation	Protection Against
SELECT-only enforcement	SQL must start with SELECT	Data modification
Keyword blocking	Blocks DROP, INSERT, UPDATE, DELETE, EXEC	SQL injection
Multi-statement blocking	Max 1 semicolon allowed	Statement chaining
Auto LIMIT	Adds LIMIT 20 if not present	Data exfiltration
Read-only DuckDB	Registered as view not table	Schema changes

Sample Queries

- What is the total revenue for each country?
- Which are the top 5 best selling products by quantity?
- What is the average order value per month?
- How many unique customers are there?
- What is the busiest hour of the day for sales?
- Who are the top 10 customers by revenue?
- Show me sales by day of week

Key Results

- Web UI URL: <http://localhost:8051>
- AI Model: Llama 3.3 70B (Groq - FREE)
- Query response time: under 2 seconds
- Security: All injection attempts blocked
- Bonus requirement: SATISFIED (+5%)

Technologies Used

- Groq API (Llama 3.3-70b-versatile) - FREE
- DuckDB for in-memory SQL execution
- Flask for web interface
- Delta Lake as data source

Final Project Summary

Complete Architecture Overview

The Real-Time Retail Analytics and Demand Prediction Platform successfully implements all 8 required layers of the Big Data Lakehouse architecture, plus the bonus natural language query interface.

Pipeline Data Flow

- Step 1: Raw CSV data loaded and cleaned (NB1)
- Step 2: Data scaled to 15.3M records for Big Data demonstration (NB2)
- Step 3: Kafka producer streams data at 2,000 records/second (NB3)
- Step 4: Delta Lake stores all data with ACID compliance on MinIO (NB4)
- Step 5: Spark processes aggregations with window functions (NB5)
- Step 6: ML models trained with R2=0.9289, tracked in MLflow (NB6)
- Step 7: Predictions evaluated and stored in Delta Lake (NB7)
- Step 8: SQL analytics with 13 queries across DuckDB and Spark (NB8)
- Step 9: Live Dash dashboard auto-refreshes every 3 seconds (NB9)
- Step 10: Flask REST API serves predictions with 100x load test passed (NB10)
- Step 11: Grafana monitors 11 metrics panels in real-time (NB11)
- Step 12: Airflow DAG orchestrates all 8 pipeline tasks every 6 hours (NB12)
- Step 13: Groq Llama 3 translates English questions to SQL (NB13 - Bonus)

Key Metrics

Metric	Value
Dataset size (scaled)	15.3 million records
Total revenue processed	GBP 10.88 million
Countries served	40
Unique customers	4,649
Unique products	4,045
Kafka throughput	2,000 records/second
ML model R2 score	0.9289 (92.89%)
ML model RMSE	87.97 units
API load test	100x requests - 100% success
Airflow tasks	8 tasks - all success in 13 seconds

Metric	Value
Grafana panels	11 live panels
SQL queries	13 (10 DuckDB + 3 Spark SQL)
Dashboard refresh	Every 3 seconds

Compliance with Project Guidelines

Requirement	Status	Evidence
Kafka streaming	DONE	NB3: 500K records at 2K/sec
MinIO + Delta Lake	DONE	NB4: retail-v2 bucket
Spark processing	DONE	NB5: 805K records, window functions
Airflow DAG	DONE	NB12: 8 tasks, 6-hour schedule
ML training	DONE	NB6: 3 models, best R2=0.9289
MLflow tracking	DONE	NB6: experiment retail-demand-prediction-v2
REST API	DONE	NB10: Flask, 4 endpoints, load tested
Grafana 3+ panels	DONE	NB11: 11 panels live
Bonus NL interface	DONE	NB13: Groq Llama 3 + DuckDB